SLE-4: Final Viva & Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM108

Name: Prathmesh Kore

Division: B

Date: 11 October 2026

GitHub / Portfolio Link (optional): https://github.com/prathmeshkore13-collab/IAI

PART A: Architecture Decision Record (ADR)

Title:

Use of Breadth-First Search (BFS) as the Primary Maze-Solving Algorithm, over Depth-First Search (DFS)

Status:

Accepted

Date:

11 October 2026

1. Context

While building the maze path-finding system across SLE-2 and SLE-3, I needed to choose a core search algorithm to find a route through a 2D grid maze from a start cell to a goal cell. The maze is represented as a grid with randomly placed walls, generated with a solvability check so a path is always guaranteed to exist between the start and goal cells. The system's Search Engine component (defined in the SLE-3 Container and Component diagrams) was deliberately built with a Frontier that could be implemented as either a queue (BFS) or a stack (DFS) without changing any other component — the Node Expander, Goal Test, Visited Checker, and get_neighbors() functions all stay identical either way, since they only depend on the Frontier exposing a consistent add/remove interface.

This meant the algorithm choice was not a question of "which one can I implement" — both were already implemented and working correctly — but "which one should the system actually rely on by default." Since the maze solver is meant to represent a usable navigation result, not just "any route, fast," I needed to decide which of these two uninformed search strategies should be the system's default, based on actual evidence rather than a guess.

2. Decision

I decided to use Breadth-First Search (BFS) as the primary, default search algorithm for the maze-solving system. Depth-First Search (DFS) is retained in the codebase as a secondary algorithm, used only for benchmarking and comparison, not as the system's default.

3. Alternatives Considered

Depth-First Search (DFS) — already implemented alongside BFS using the same Frontier/Node Expander structure; faster in testing but does not guarantee the shortest path.

Greedy Best-First Search / A* with a Manhattan-distance heuristic — would likely be faster than BFS while still being optimal (A*), but requires designing and validating a heuristic function, which was outside the uninformed-search comparison scope of SLE-2.

Bidirectional BFS — could reduce node expansions by searching from both start and goal simultaneously, but adds real implementation complexity (two frontiers, meeting-point detection) that wasn't justified for this project's maze sizes.

4. Rationale / Justification

The SLE-2 profiling results (3 maze sizes, 5 timed runs each) directly support this decision. BFS found

the true shortest path in all 3 test cases, while DFS never found the optimal path even once — 2.0x to 2.4x longer every time. This matches the theoretical guarantee that BFS is complete and optimal on an unweighted graph, while DFS has no such guarantee. The table below summarizes the measured numbers this decision is based on:

Maze Size	BFS Time (ms)	DFS Time (ms)	BFS Path	DFS Path	Path Ratio
20x20 (best case)	0.2969	0.1765	39	77	1.97x longer
40x40 (average case)	1.0254	0.7718	79	167	2.11x longer
70x70 (worst case)	4.2612	2.8732	139	333	2.40x longer

DFS was consistently faster in wall-clock time and expanded fewer nodes (roughly 54-56% of BFS's node count at every size), but that speed came directly at the cost of path quality — a path 2.4x longer than necessary is a serious defect for a system meant to represent a usable route, not just a fast computation. Importantly, the gap in path quality widened as the maze got larger (1.97x at 20x20, growing to 2.40x at 70x70), suggesting the problem would only get worse at larger, real-world scale, not better.

The SLE-2 py-spy flame graph adds independent confirmation of where the time actually goes: BFS's share of CPU time (~65-69% across two separate profiling runs) closely tracked its share of total node expansions, meaning the extra time BFS spent was proportional to genuinely necessary extra work, not hidden inefficiency or waste elsewhere in the implementation.

From the SLE-3 design side, choosing BFS as the default carried zero architectural cost: the Frontier component in the Search Engine was already built to accept either a deque or a stack, so this decision only meant choosing BFS as the default data structure passed to the Frontier — no component needed to be restructured or rewritten, and the Neighbour Generator, Visited Checker and Goal Test components required no changes at all.

Given that the system's stated purpose is path-finding for a maze — i.e., producing a usable, trustworthy route rather than merely a fast computation — correctness of the result outweighs a modest, consistent 1.3x-1.7x speed penalty measured across all three test cases.

5. Consequences

Positive:

- Always returns the true shortest path, on every maze size tested — a measurable, provable guarantee rather than a probabilistic one.
- Predictable worst-case path quality, which matters for any downstream use (e.g. real navigation) built on top of this search.
- Easy to justify and defend with concrete evidence — the same SLE-2 numbers and SLE-3 diagrams support this decision directly.

Negative / Trade-offs:

- Higher node-expansion and memory cost: 3,617 nodes expanded on the 70x70 maze vs DFS's 1,951 — BFS's frontier must hold an entire exploration "layer" in memory at once (O(b^d) space).
- Consistently 1.3x-1.7x slower in wall-clock time than DFS across all three tested maze sizes.
- Does not scale as well as DFS on much larger or deeper mazes; if maze size grows significantly (e.g. 500x500), BFS's memory use could become the real bottleneck, at which point an informed search like A* would likely be the better next decision.

6. References

- GitHub repository: https://github.com/prathmeshkore13-collab/IAI
- SLE-2 profiling report and raw results: SLE2_25UAM108_PrathmeshKore.docx, results.json

SLE-3 C4 diagrams referenced in this ADR: container_diagram.png, component_diagram.png

PART B: Short Summary of My SLE Journey

SLE-1— AI-Augmented Workflow:

I set up an AI-assisted coding workflow from scratch — installing Git, configuring GitHub, and using Claude to generate a simple chatbot agent (agent.py). Every AI-generated section was logged in a Contribution Log, distinguishing it from the parts I wrote or fixed myself (such as a missing docstring and a renamed duplicate file), alongside a README and a proper Git commit history.

SLE-2 — Empirical Performance Analysis:

I moved from "does it work" to "how well does it work." I implemented BFS and DFS on randomly generated grid mazes across three sizes (20x20, 40x40, 70x70), measured real execution time and nodes expanded using time.perf_counter, and used py-spy — a real sampling profiler, not a theoretical estimate — to capture a genuine CPU flame graph. That flame graph confirmed BFS's runtime share matched its node-expansion share almost exactly, which became important supporting evidence for this ADR.

SLE-3 — C4 Architecture Diagrams:

I extended the same maze project into a four-level C4 architecture model. The Container diagram shows the Maze Input, Search Engine, Neighbour Generator, Visited Set, Experiment Runner and Output modules and how they connect; the Component diagram zooms into the Search Engine itself, showing the Frontier, Node Expander, Goal Test, Visited Checker and Path Builder — and reflects how my actual code works (building the path inline rather than reconstructing it from parent pointers afterward).

SLE-4 — This ADR:

This document brings SLE-2 and SLE-3 together: using the concrete numbers from SLE-2 and the component design from SLE-3, I formally justify why BFS was chosen as the default search algorithm over DFS, and this ADR is what I will defend in the viva.

PART C: AI Contribution Note

AI tools used: Claude (Anthropic).

What AI helped with: Drafting the ADR structure and wording; organizing the SLE-2 numbers and SLE-3 component references into a coherent Rationale section, including building the summary comparison table; formatting this document to match the required template; and helping structure the Part B journey summary across all four SLEs.

What I did myself: Made the underlying decision while actually building and running the SLE-2 experiments — choosing to compare BFS vs DFS at all, choosing the three maze sizes and wall densities used for testing; ran all profiling scripts and py-spy captures myself and checked the raw output before trusting the numbers; reviewed the SLE-3 diagrams against my own code to confirm they were accurate; and I am the one defending this decision in viva using my own understanding of the code and results, not memorized text.

Where I would push back if asked: if a reviewer suggested A* was simply "better," I would point out that A* requires a heuristic (e.g. Manhattan distance) that still needs to be designed and validated — it wasn't a zero-cost upgrade available for free within this project's scope, which is why it is listed as an alternative rather than the chosen option.